

לוח שידוע בן כמה הוא

שבוע: 11 ניקוד: 100 מועד הגשה: לפני תחילת שיעור 13 משקל בציר הסופי: מטלת בית · ארבע מטלות בית הן 10 אחוזים מהציון זמן משוער: כשלוש שעות עבודה

מטרה

יישום אחד קטן שמחבר את שלושת השבועות האחרונים: `fetch` מביא, `JSON` ו-`localStorage` שומרים, והמטמון יודע בן כמה הוא — ואומר את זה למשתמש.

שש ערים, מדידת מזג אוויר נוכחית לכל אחת. העמוד נפתח בלי לגעת ברשת: מה שנשמר מוצג מיד, ורק אם ביקשת נשלחת בקשה. זה לא קיצור דרך. יישום שהציור הראשון שלו מחכה לשש בקשות מציג מלבן לבן על חיבור גרוע ושום דבר בלי חיבור — ועם מטמון הוא יכול להציג שש מדידות אמיתיות, מיד, גם ברכבת.

מה חדש כאן, מעבר לשבוע 11

מטמון הוא לא "לשמור את התשובה כדי לא לשאול שוב". זה מסד נתונים בלי שום דרך לטעות, וזו הדרך שבה יישום מציג למישהו את הטמפרטורה של אתמול כאילו היא עכשיו.

מטמון הוא שתי עובדות שנשמרות יחד: התשובה, ומתי היא הייתה נכונה. כל השאר נובע מהשנייה:

המצב	מה זה אומר	ומה עושים
טרי	בתוך זמן החיים	משתמשים, בלי לומר כלום
ישן	מחוץ לזמן החיים	משתמשים אם צריך — ואומרים את זה
חסר	לא נשאל, או שהמפתח נוקה	אין מה להציג. זה <code>idle</code> , לא "לא נמצא כלום"

תוצרי למידה

- לכתוב שכבת אחסון שמאחסנת ערך וחותרת זמן ויודעת לגזור מהם גיל.
- לכתוב שכבת רשת שכל תשובה בה נבדקת ולכל בקשה בה יש תקרת זמן.
- להציג חמישה מצבים לכל שורה, ולהוכיח ששניים מהם לא יכולים להיות נכונים יחד.
- להשתמש ב-`Promise.allSettled` במקום שבו `Promise.all` הוא התשובה הלא נכונה, ולהסביר למה.
- להיות כן עם המשתמש לגבי מקור הנתונים וגילם — וזו הדרישה שהמטלה נמדדת בה.

מה יש בתיקייה

קובץ	מה עושים איתו
<code>js/cache.js</code>	הקובץ הראשון שאתה כותב. שלד עם חוזה
<code>js/api.js</code>	הקובץ השני. אותו דבר

קובץ	מה עושים איתו
js/app.js	נתון, עם ארבעה סימוני CODE HERE
js/cities.js	נתון. אל תשנה אותו. שש הערים וקודי מזג האוויר
index.html	נתון וגמור. לא נוגעים בו
styles.css	נתון וגמור. אך נקודה במטלה אינה על עיצוב

מגבלות – מה אסור השבוע

מריצים עם Live Server. הקבצים כאן מודולים, ומודול לא נטען מעל file://.

אל תיגע ב-js/cities.js. הבדוק מייבא אותו כמו שהוא, ושש הערים בסדר הזה הן חלק מהחוויה. עיר שביעית היא לא בונס, היא בדיקה שנכשלת.

js/cache.js לא נוגע ברשת. js/api.js לא נוגע ב-localStorage. שתי השכבות לא מכירות זו את זו, ואת שתיהן מחבר רק js/app.js.

אף אחד משניהם לא נוגע ב-DOM. אין document בשניהם.

js/render – כלומר הציור שבתוך js/app.js – נשאר סינכרוני. אין await בתוכו.

העמוד לא שולח בקשה בטעינה. זו דרישה, לא מגבלה.

localStorage.clear() אסור. הוא מוחק את כל ה-origin — כלומר גם את כל הפרויקטים האחרים שלך על אותו פורט.

ליבה • 70 נקודות ליבה

js/cache.js • 30 נקודות

1. readAll() מחזירה את המפה השמורה, או {}. היא לא זורקת לעולם — היא רצה לפני הציור הראשון, וזריקה שם היא עמוד לבן.
2. writeOne(cityId, entry) כותבת ערך אחד ומשאירה את השאר. מחזירה האם נכתב.
3. clearCache() מוחקת את המפתח של היישום הזה. לא localStorage.clear().
4. isEntry — בדיקת מבנה. JSON תקין אינו נתונים תקינים, ורשומה בלי fetchedAt היא לא רשומת מטמון — היא שמועה.
5. migrate — שלוש תשובות: המעטפה הנוכחית, מפה חשופה מגרסה 0 שמאמצים, וכל השאר.
6. freshness(entry, now) — ארבע שורות, והן הלב של המטלה.

למה now הוא פרמטר ולא Date.now() בפנים

כי ככה הפונקציה טהורה, ופונקציה טהורה של שני מספרים אפשר לבדוק בלי לחכות עשר דקות. הבודק בודק אותה בדיוק ככה, ואתה יכול גם.

ושים לב לרשומה שחתומה בעתיד — מישו שהזיז את השעון אחורה. זו לא שגיאה בקוד שלך, ו-NaN במקום מספר דקות על המסך — כן.

למה `fetchAt` הוא מספר ולא `Date`

שבוע 10, מטלה אחת אחר כך. `JSON.stringify(new Date())` מייצר מחרוזת ISO, ול-`JSON.parse` אין מושג שזה היה פעם משהו אחר – אז יישום שמחזיק `Date` בזיכרון עובד מצוין עד הרענון הראשון, ואז `entry.fetchAt.getTime is not a function` נזרק מתוך `render`. `Date.now()` הוא מספר. הוא שורד את המסע, הוא מחסיר בלי טקסים, ואין מה לשכוח.

js/api.js · 25 נקודות

1. `fetchWeather(city, { signal })` – חמישה שלבים, וכולם משבוע 11: `new URL`, `AbortSignal.any` עם פסק זמן · `res.ok` · **בשורה הבאה** · `res.json()` ב-`try` משלו · נרמול.
2. `toReading` לוקח את שלושת השדות שהיישום מצייר, וחותם `fetchAt`. לא יותר: כל שדה שאתה מעתיק לנתונים שלך הוא שדה שהתחייבת לתחזק, והוא של מישהו אחר.
3. `AbortError` נזרק הלאה כמו שהוא, ולא נעטף ב-`ApiError`.
4. כל תקלה אחרת היא `ApiError` עם `kind` ועם **משפט שלם בעברית**.

חמשת המצבים · 15 נקודות

לכל שורה יש `data-state`, ותמיד בדיוק אחד:

המצב	מתי
<code>idle</code>	אין מדידה שמורה, ואין בקשה
<code>loading</code>	בקשה פתוחה לעיר הזאת
<code>fresh</code>	שמור, בתוך זמן החיים
<code>stale</code>	שמור, מחוץ לזמן החיים
<code>error</code>	הניסיון האחרון נכשל

1. **`error` ו-`stale` מציגים את המדידה השמורה בכל זאת** – ואומרים שהיא לא מעודכנת. "לא הצלחנו להגיע לשירות, וזה מה שהוא אמר לפני אחת עשרה דקות" שימושי הרבה יותר מכרטיס ריק, **כל עוד שני החצאים נאמרים**. מספר ישן בלי תווית הוא שקר עם חותמת זמן על כלום.
2. `#summary` אומר כמה מהלוח אמיתי וכמה ישן.
3. `#storage-note` מופיע רק כשכתיבה נכשלה.

הרחבה · 20 נקודות

הרחבה

1. `#refresh-all` מרענן את כל השש בבת אחת – `Promise.allSettled`, לא `Promise.all`, ולא שישה `await` בשורה. תדע להסביר את שתי הבחירות.
2. **לחיצה שנייה על `#refresh-all` מבטלת את הסבב הקודם**. כל הביטולים האלה מגיעים כ-`AbortError`, ואף שורה לא צריכה להאדים בגללם.

3. **השעון מתקתק.** "טרי" נגזר מהזמן הנוכחי, ולשונית שנשארה פתוחה אחת עשרה דקות מציגה תווית שהפסיקה להיות נכונה בלי ששום דבר קרה. **שורה אחת.**
4. **#clear-cache שואל קודם.** זה הדבר היחיד כאן שאי אפשר לבטל.

אתגר · 10 נקודות אתגר

מטמון שמתרענן מעצמו, בלי לחסום. בטעינה, אחרי שהעמוד כבר צויר מהמטמון:

- הצג את מה שיש — **מיד**, כולל אם הוא ישן.
 - ואז, ורק ואז, רענן ברקע את הערים שהן **stale**.
 - המשתמש רואה נתונים בזמן שהם מתעדכנים**, ולא ספינר במקומם.
- זה נקרא **stale-while-revalidate**, וזו התבנית שכל אפליקציה טובה שהשתמשת בה בנויה עליה. היא גם הסיבה שהמטמון קיים: לא לחסוך בקשות, אלא **לא לגרום למשתמש לחכות בשביל משהו שכבר יש לך**.

מה נשבר אם תעשה את זה לא נכון

אם הרענון ברקע מציב **loading** על שורה שיש בה נתונים — הרווחת ספינר במקום מספר, וזה בדיוק מה שהתבנית קיימת כדי למנוע. השורה צריכה להישאר **stale** ולהתעדכן כשהתשובה מגיעה.

מדיניות AI לשבוע הזה

שותף מימוש. אותה אזהרה כמו בשבוע 11, ועוד אחת שמיוחדת למטלה הזאת.

קוד מטמון הוא הדבר ש-Claude כותב הכי מהר וש-Claude שוכח בו את אותו דבר בכל פעם: את חותמת הזמן.
`localStorage.setItem(key, JSON.stringify(data))` הוא מטמון בלי גיל, וכל היישום הזה הוא הגיל.

שלוש טעויות שהוא עושה כאן שוב ושוב:

1. `fetchAt: new Date()` — שורד את הכתיבה ולא את הקריאה.
 2. `Date.now()` בתוך **freshness** במקום כפרמטר, מה שהופך אותה לבלתי בדיקה.
 3. `Promise.all` ב-**refreshAll**, כי הוא קצר יותר — וכי הוא לא שאל מה קורה כשעיר אחת נופלת.
- תבקש ממנו במפורש גם את מצב stale וגם את מצב error שמציג נתונים ישנים**, ואז קרא את הפלט מול הטבלה של חמשת המצבים.

הגשה

תיקייה `homework/hw-4/` במאגר הסמסטר שלך.

הגשה במודל: כתובת המאגר + ה-SHA של הקומיט.

הגשה מאוחרת: ההגשה נסגרת במועד שכתוב במודל. הגשה שמגיעה אחרי המועד מתקבלת ונבדקת במלואה, ומסומנת כמאוחרת — היא אינה מורידה נקודות.

- ☐ . פתחת את העמוד בפעם הראשונה: שש שורות `idle` , ואפס בקשות ברשת (בדוק ב-Network).
- ☐ . לחצת "רענן" על עיר אחת: `loading` ואז `fresh` , עם טמפרטורה.
- ☐ . רעננת את העמוד: המדידה שם **מיד** , ועדיין אפס בקשות.
- ☐ . "רענן הכול": שש שורות מתעדכנות, ולא אחת אחרי השנייה.
- ☐ . כיבית את ה-Wi-Fi ולחצת "רענן": השורה `error` , המדידה עדיין מוצגת, וכתוב שהיא לא מעודכנת.
- ☐ . פתחת Application, ערכת `fetch` של אחת הערים לערך ישן, ורעננת: היא `stale` .
- ☐ . השחתת את הערך של המפתח ביד ורעננת: העמוד עלה.
- ☐ . שמת `null` בערך ורעננת: העמוד עלה.
- ☐ . "נקה מטמון" שאל קודם, ואחרי אישור הכול חזר ל- `idle` .
- ☐ . המילה `fetch` לא מופיעה ב- `js/cache.js` , והמילה `localStorage` לא ב- `js/api.js` .